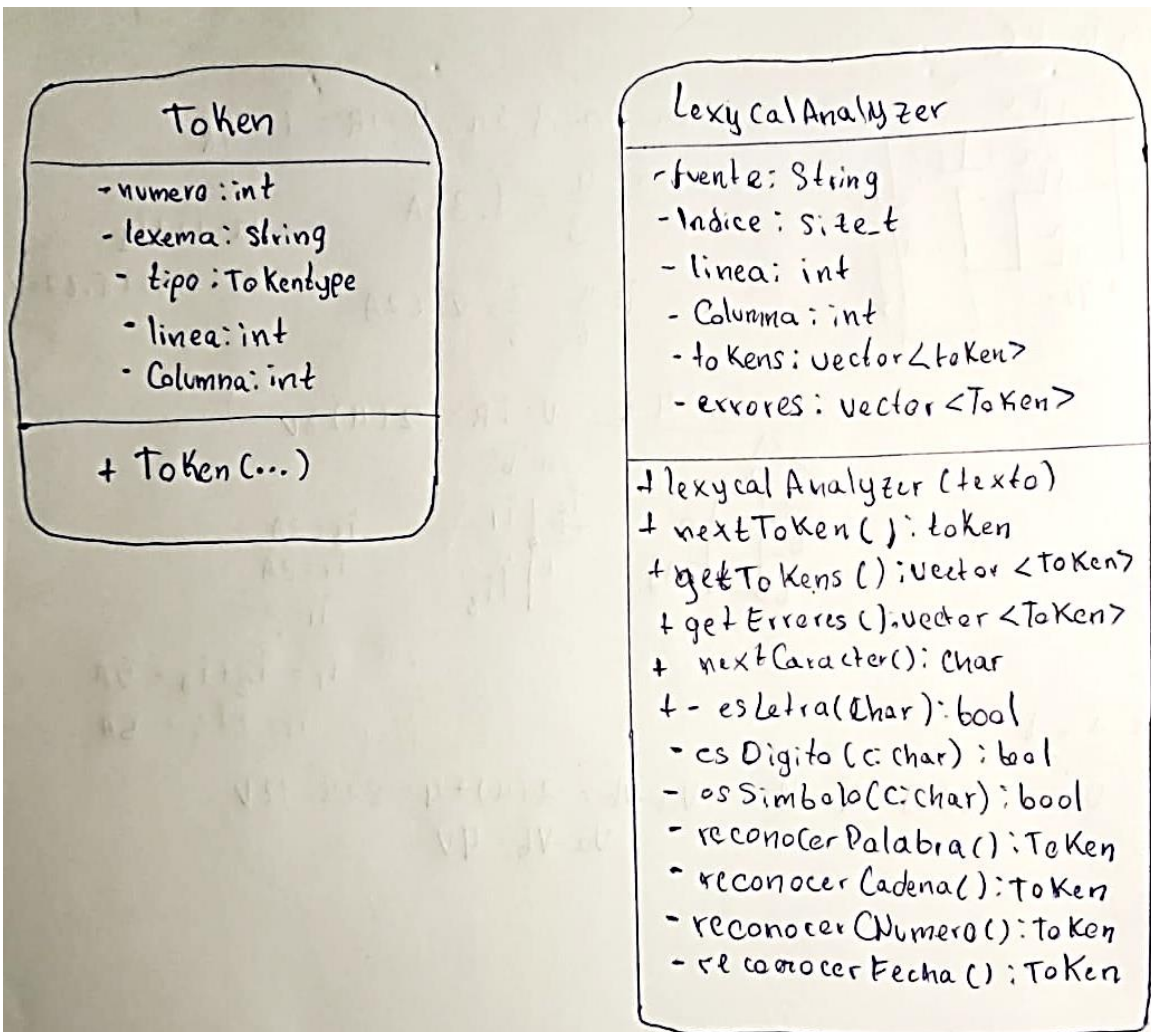


MANUAL TECNICO

DIAGRAMA UML



1

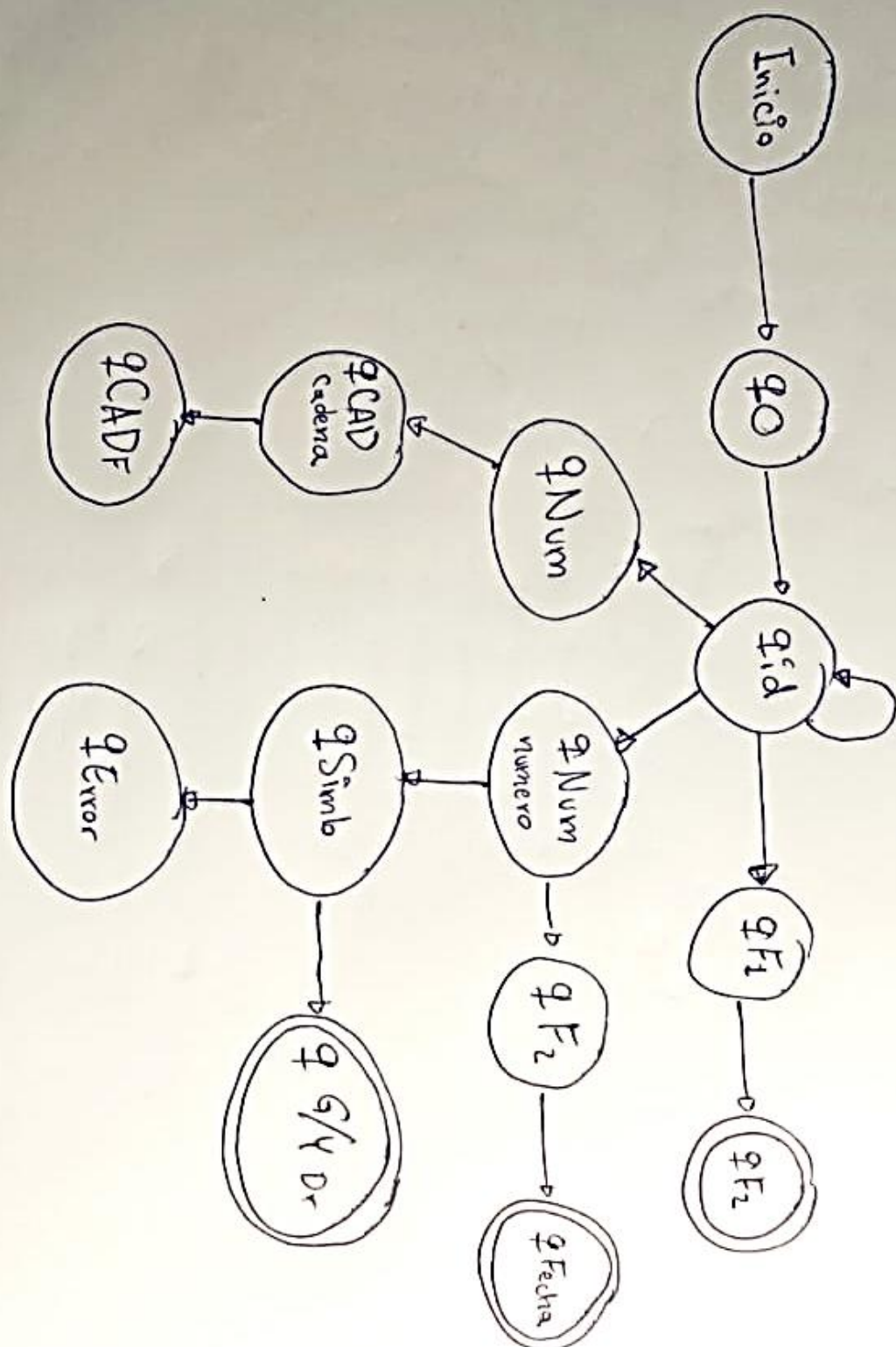


DIAGRAMA AFD

RESUMEN DE PARSER:

Tipo de parser: descendente recursivo (top-down), porque la gramática está diseñada para ser clara y sin ambigüedad.

Entrada: secuencia de tokens generados por el analizador léxico.

Proceso:

1. El parser comienza en el símbolo inicial <PROGRAMA>.
2. Verifica que la secuencia siga las reglas de la gramática (ej. TABLERO, luego una cadena, luego {}).
3. Construye un árbol de derivación o árbol sintáctico que representa la estructura del tablero.
4. Si encuentra un token inesperado, genera un error sintáctico y lo acumula en la tabla de errores.

Salida:

- Árbol sintáctico válido (para semántica y generación de reportes).
- Tabla de errores sintácticos si la entrada no cumple la GLC.

GLC

$\langle \text{programa} \rangle ::= \text{Tablero} \langle \text{Cadena} \rangle \{ \langle \text{lista_columna} \rangle \}$

$\langle \text{lista_columna} \rangle ::= \langle \text{columna} \rangle \langle \text{lista_columnas} \rangle \mid \epsilon$

$\langle \text{columna} \rangle ::= \text{columna} \langle \text{Cadena} \rangle \{ \langle \text{lista_tareas} \rangle \} \{ \}$

$\langle \text{lista_tareas} \rangle ::= \langle \text{tarea} \rangle \langle \text{lista_tareas} \rangle \mid \epsilon$

$\langle \text{tarea} \rangle ::= \text{tarea} \{ \} \langle \text{Cadena} \rangle \{ \} \langle \text{atributos} \rangle \{ \}$

$\langle \text{atributos} \rangle ::= \langle \text{atributo} \rangle \{ \} \langle \text{atributos} \rangle \mid \langle \text{atributo} \rangle$

$\langle \text{atributo} \rangle ::= \text{prioridad} \{ \} \langle \text{prioridad} \rangle \mid \text{responsable}$

$\{ \} \langle \text{Cadena} \rangle$

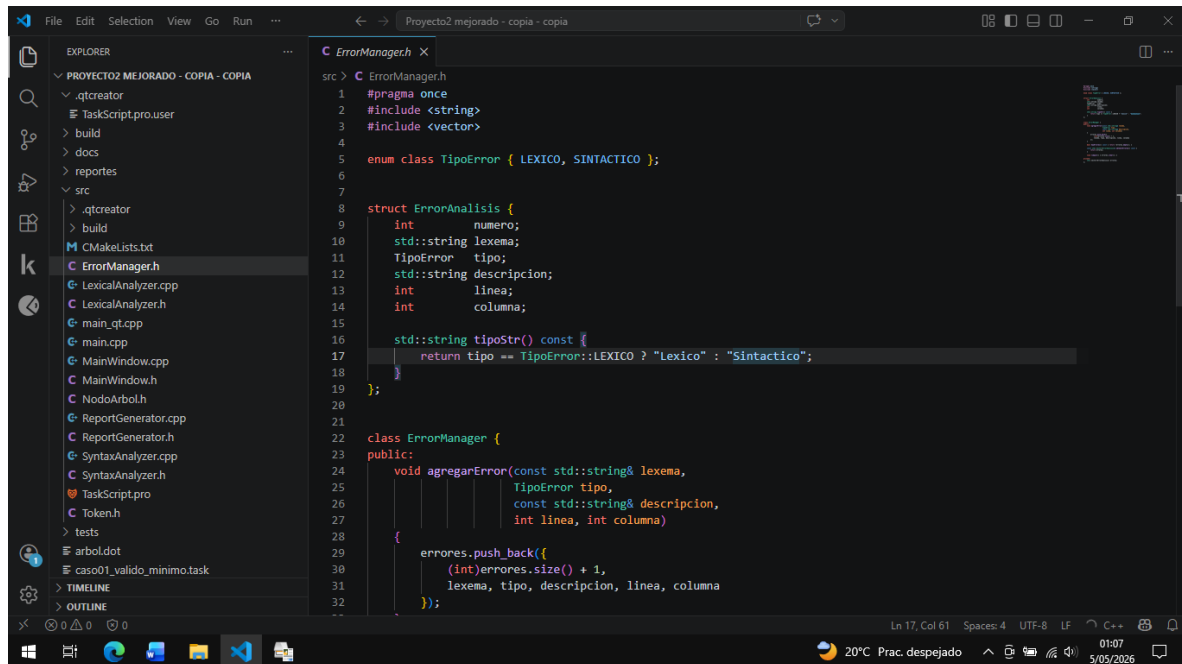
$\mid \text{fecha_limite}$

$\{ \} \langle \text{fecha} \rangle$

$\langle \text{prioridad} \rangle ::= \text{ALTA} \mid \text{MEDIA} \mid \text{BAJA}$

Error manager. H

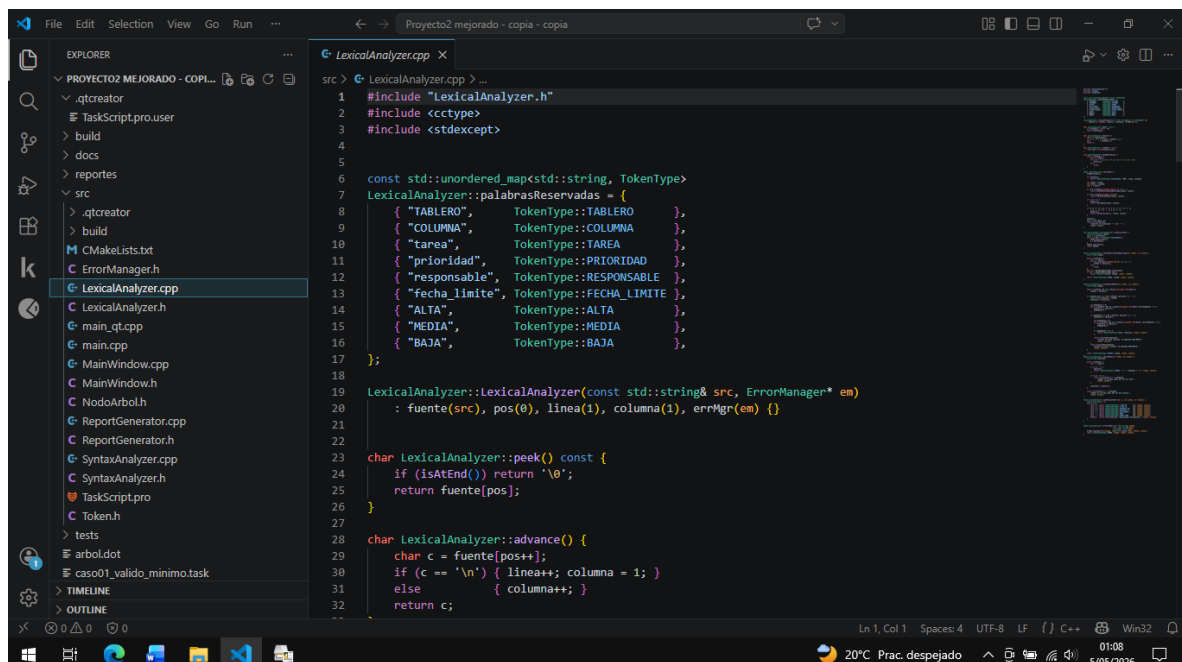
En este header definimos la estructura del c++.



```
1 #pragma once
2 #include <string>
3 #include <vector>
4
5 enum class TipoError { LEXICO, SINTACTICO };
6
7
8 struct ErrorAnalysis {
9     int numero;
10    std::string lexema;
11    TipoError tipo;
12    std::string descripcion;
13    int linea;
14    int columna;
15
16    std::string tipoStr() const {
17        return tipo == TipoError::LEXICO ? "Lexico" : "Sintactico";
18    }
19 };
20
21
22 class ErrorManager {
23 public:
24     void agregarError(const std::string& lexema,
25                     TipoError tipo,
26                     const std::string& descripcion,
27                     int linea, int columna)
28     {
29         errores.push_back({
30             (int)errores.size() + 1,
31             lexema, tipo, descripcion, linea, columna
32         });
33     }
34 };
```

Analizador Lexico

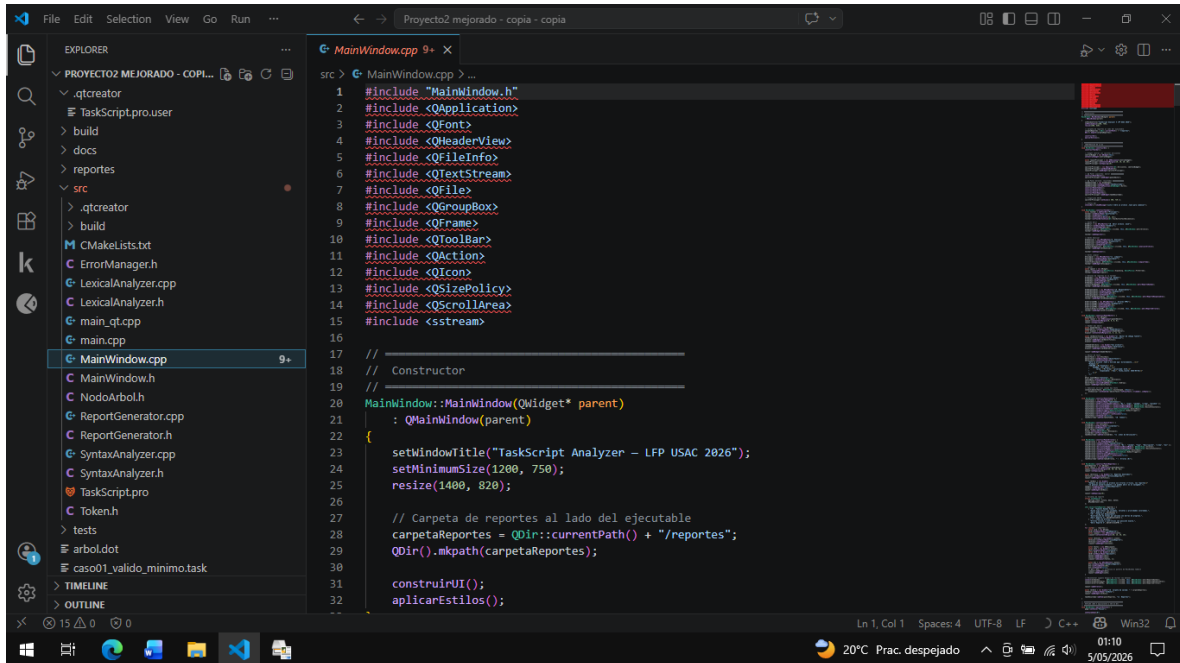
El analizador léxico definimos los tokens, dentro del header definimos la estructura de las clases que serán definidas en el archivo C. aquí definimos los tokens tablero, columna, tarea, prioridad, responsable, alta, media, baja.



```
1 #include "LexicalAnalyzer.h"
2 #include <cctype>
3 #include <string>
4
5
6 const std::unordered_map<std::string, TokenType>
7 LexicalAnalyzer::palabrasReservadas = {
8     { "TABLERO", TokenType::TABLERO },
9     { "COLUMNA", TokenType::COLUMNA },
10    { "TAREA", TokenType::TAREA },
11    { "prioridad", TokenType::PRIORIDAD },
12    { "responsable", TokenType::RESPONSABLE },
13    { "fecha_limite", TokenType::FECHA_LIMITE },
14    { "ALTA", TokenType::ALTA },
15    { "MEDIA", TokenType::MEDIA },
16    { "BAJA", TokenType::BAJA },
17 };
18
19 LexicalAnalyzer::LexicalAnalyzer(const std::string& src, ErrorManager* em)
20 : fuente(src), pos(0), linea(1), columna(1), errMgr(em) {}
21
22 char LexicalAnalyzer::peek() const {
23     if (isAtEnd()) return '\0';
24     return fuente[pos];
25 }
26
27 char LexicalAnalyzer::advance() {
28     char c = fuente[pos++];
29     if (c == '\n') { linea++; columna = 1; }
30     else { columna++; }
31     return c;
32 }
```

Mainwindows:

La estructura principal de la interfaz a usar en qt, el main Windows lleva la estructura principal de la interfaz.



```
1 #include "MainWindow.h"
2 #include <QApplication>
3 #include <QFont>
4 #include <QHeaderView>
5 #include <QFileInfo>
6 #include <QTextStream>
7 #include <QFile>
8 #include <QGroupBox>
9 #include <QFrame>
10 #include <QToolBar>
11 #include <QAction>
12 #include <QIcon>
13 #include <QSizePolicy>
14 #include <QScrollArea>
15 #include <sstream>
16
17 //
18 // Constructor
19 //
20 MainWindow::MainWindow(QWidget* parent)
21 : QMainWindow(parent)
22 {
23     setWindowTitle("TaskScript Analyzer - LFP USAC 2026");
24     setMinimumSize(1200, 750);
25     resize(1400, 820);
26
27     // Carpeta de reportes al lado del ejecutable
28     carpetaReportes = QDir::currentPath() + "/reportes";
29     QDir().mkpath(carpetaReportes);
30
31     construirUI();
32     aplicarEstilos();
33 }
```